

NEBULA

Cahier des charges : infrastructure

Déploiement et exploitation d'un réseau social sur un cluster Docker Swarm de trois machines.

Rubrique	Valeur
Module	Clusterisation de conteneurs
Orchestrateur imposé	Docker Swarm (Docker Engine 29.x)
Nature du projet	Infrastructure. La part de développement applicatif est volontairement minimale.
Infrastructure	Trois machines virtuelles que vous créez et administrez vous-mêmes.
Travail	Individuel
Rendu	Dépôt Git accessible à l'enseignant, plus une démonstration en direct
Soutenance	10 minutes par étudiant, manipulations en direct comprises
Date de rendu	à préciser en séance

1. Contexte et enjeux

Nebula est un réseau social en forte croissance. L'application évolue vite, les pics de trafic sont imprévisibles, et l'équipe technique doit pouvoir publier des versions fréquemment sans rupture de service.

Le périmètre de ce document est l'infrastructure. L'application n'est qu'un support minimal permettant de valider le routage, la communication entre services, la persistance, le traitement asynchrone, les mises à jour progressives et le retour arrière. Le design, l'interface et la complétude fonctionnelle sont hors sujet.

Vous livrez une plateforme exploitable : n'importe qui doit pouvoir la déployer, l'observer et la dépanner en suivant votre documentation. Nebula doit survivre aux incidents courants : machine indisponible, image défectueuse, saturation ponctuelle, composant interne en erreur.

Les quatre qualités évaluées

- Fiabilité : le système reste utilisable pendant la mise à jour d'un service sans état.
- Opérabilité : diagnostic et retour arrière sont réalisables rapidement, et documentés.
- Sécurité minimale : secrets hors dépôt, exposition réduite au strict nécessaire, accès contrôlé aux outils d'administration.
- Traçabilité : on identifie précisément ce qui tourne, en quelle version, sur quelle machine, en combien d'instances.

Le risque réel. Les architectures microservices échouent rarement par manque de fonctionnalités. Elles échouent quand on ne peut plus déployer, observer, isoler ni restaurer proprement. Ce projet est évalué sur ces quatre verbes.

2. Périmètre : ce qui est demandé, ce qui ne l'est pas

Ce point conditionne votre organisation. Lisez-le avant de commencer.

	Attendu	Non attendu
Infrastructure	Création des trois machines, cluster, réseaux, placement, secrets, mises à jour progressives,	Rien n'est retiré ici : c'est l'intégralité de la note.

	Attendu	Non attendu
	retour arrière, registry, livraison automatisée, procédures d'exploitation.	
Application	Une implémentation de référence des contrats de l'annexe A vous est fournie. Vous pouvez l'utiliser telle quelle, la modifier ou la réécrire : cela ne rapporte ni ne coûte aucun point.	Interface graphique, authentification réelle, fil algorithmique, modération, gestion de médias.
Volume de code	Le langage est libre. Visez le strict nécessaire pour que les contrats répondent.	Tests unitaires, couverture, qualité du code applicatif.

Conséquence directe. Un étudiant qui passe plus de deux heures à écrire du code applicatif s'est trompé de projet. Le temps doit aller dans la construction du cluster, les fichiers de déploiement, les procédures et la répétition de la démonstration.

3. Architecture attendue

Un point d'entrée HTTP, des services applicatifs sans état, et des dépendances d'infrastructure. Les sept services ci-dessous sont imposés. Les technologies signalées « libre » sont à choisir et à justifier.

Service	Rôle	Technologie	Enjeu d'exploitation
edge	Point d'entrée HTTP, routage vers les services	nginx ou Traefik	Seul service publié. Montée en charge horizontale.
comptes	Création et lecture de comptes	libre	Sans état, répliquable, mise à jour progressive
publications	Publication et lecture du fil	libre	Sans état, appelle comptes et produit un événement
worker-medias	Consomme les événements, écrit un objet	libre	Traitement asynchrone, rejouabilité
db	Données durables	PostgreSQL 18	Persistance, placement contraint, sauvegarde
cache	Cache de lecture du fil	Redis	Données volatiles, aucune sauvegarde nécessaire
bus	File d'événements	RabbitMQ ou NATS	Découplage, visibilité des messages en erreur

Sur cache et bus. Ces deux briques ne demandent aucune compétence nouvelle : ce sont des services Swarm comme les autres, avec une image, un réseau interne, un secret et une sonde de santé. Toute la difficulté est infrastructure.

4. Exigences non fonctionnelles

Ces exigences sont prioritaires sur toute sophistication applicative.

- Disponibilité pendant les mises à jour : la mise à jour d'un service sans état ne doit pas rendre Nebula inutilisable.
- Montée en charge : le nombre d'instances d'un service sans état doit pouvoir augmenter, avec répartition effective et preuve de cette répartition.
- Segmentation : les composants internes ne sont pas accessibles publiquement. Un seul port est publié sur le cluster.

- Traçabilité : toute version déployée est identifiable par un tag d'image immuable. Le tag latest est interdit.
- Secrets : aucune donnée sensible en clair dans le dépôt, ni dans l'historique Git.
- Restauration : une sauvegarde de la base est produite, et une restauration est effectivement testée une fois.

5. Périmètre applicatif minimal

Le code n'est pas l'objectif. Il doit seulement permettre de démontrer quatre choses, et rien de plus.

- Chaque service applicatif expose une route de santé sur /health.
- Un flux entre services : publications appelle comptes pour vérifier l'auteur.
- Une écriture puis une lecture persistante, prouvant que la donnée survit à un redémarrage complet du cluster.
- Un traitement asynchrone : publications produit un événement, worker-medias le consomme.

Les contrats HTTP et le format de l'événement sont donnés en annexe A, et une implémentation de référence vous est fournie. Vous pouvez la remplacer par la vôtre si vous préférez : aucune liberté de conception n'est attendue sur ce point, et aucune n'est valorisée.

6. Le cluster : trois machines virtuelles

C'est le premier travail du projet, et le socle de tout le reste. Vous créez vous-mêmes trois machines virtuelles sur votre poste, vous y installez Docker, et vous les reliez en un seul cluster Swarm.

Une seule machine ne suffit pas : sans plusieurs nœuds, ni le placement, ni la répartition entre machines, ni la migration de tâches ne sont démontrables, et quatre des scénarios de vérification deviennent injouables.

Ce qu'il faut obtenir

Étape	Résultat attendu
Créer	Trois machines virtuelles Linux, nommées de façon lisible, avec Docker installé et un accès en ligne de commande.
Relier	Un gestionnaire, deux exécutants, tous visibles dans une même liste de nœuds, tous à l'état prêt et actif.
Étiqueter	Des étiquettes de nœuds posées, qui serviront à contraindre le placement de la base et du bus.
Éteindre	Les trois machines arrêtées proprement, l'une après l'autre.
Rallumer	Les trois machines redémarrées, le cluster reformé tout seul, les services revenus, les données toujours là.
Prouver	Une sortie de commande qui montre les trois machines dans le même cluster, et la machine qui dirige.

Les points qui coûtent du temps si on les néglige

- Les échanges entre nœuds utilisent trois canaux : le port 2377 en TCP pour l'administration, le port 7946 en TCP et UDP pour la découverte, et le port 4789 en UDP pour le réseau applicatif. Si le dernier est bloqué, vos services démarreront normalement sans jamais se voir. Testez-le avant d'écrire une ligne de déploiement.
- L'adresse annoncée au moment de former le cluster doit être joignable depuis les autres machines. Une adresse locale ne convient pas : chaque machine a la sienne.
- Si vos machines prennent une adresse différente à chaque démarrage, le cluster ne se reformera pas. Fixez les adresses, ou documentez la procédure de reprise.
- Trois machines sur un seul poste, plus sept services : déclarez des limites de ressources, sinon le système tue des conteneurs au hasard et vous chercherez une cause applicative qui n'existe pas.
- Topologie à justifier : trois nœuds autorisent un gestionnaire et deux exécutants, ou trois gestionnaires. Écrivez lequel vous avez choisi et pourquoi, en vous appuyant sur ce que vous savez du quorum.

Le scénario qui tombe à la soutenance. Vous éteignez vos trois machines devant l'intervenant, puis vous les rallumerez. Le cluster doit se reformer, les services redémarrer et les données être toujours là, sans intervention manuelle autre que celle décrite dans votre documentation. Préparez-le, chronométrez-le, et sachez expliquer ce qui se passe pendant ce temps.

Si la virtualisation n'est pas disponible sur votre poste, une solution de remplacement à trois nœuds Docker imbriqués est acceptée. Elle couvre les mêmes exigences, à l'exception du diagnostic réseau de bas niveau. Signalez-le dans votre documentation.

7. Exigences Docker Swarm

Domaine	Exigence
Cluster	Trois nœuds, tous rattachés. Topologie et rôles documentés et justifiés. Arrêt et redémarrage complets maîtrisés.
Réseaux	Au minimum deux réseaux : un pour le flux public, un pour les échanges internes. La base et le bus ne sont joignables que depuis le réseau interne.
Données	Volume persistant pour la base, placement contraint sur un nœud étiqueté. Procédure de sauvegarde et de restauration décrite et exécutée.
Mises à jour	Paramètres de mise à jour progressive sur les services sans état, et politique de retour arrière définie. Le comportement en cas d'échec est explicite.
Sécurité	Secrets pour tout identifiant. Un seul port publié. Accès à l'outil d'administration protégé par mot de passe et restreint.
Traçabilité	Images taguées par version et par empreinte de commit. La version en cours est identifiable depuis le cluster.

8. Industrialisation : registry et livraison

- Les images sont récupérables par les trois machines, et pas seulement par celle qui les a construites.
- Le registry ne vit pas dans le cluster : pour redémarrer le cluster il faut tirer les images, et pour tirer les images il faudrait que le cluster tourne. Installez-le à côté.
- La fabrication et la publication des images sont automatisées : un enchaînement construit, tagué et publié sans intervention manuelle.
- Le déploiement sur le cluster est déclenchable de manière contrôlée, et non à chaque commit.
- Les identifiants de registry et de déploiement sont protégés, jamais présents dans le dépôt.

Règle à respecter. L'enchaînement d'intégration produit l'artefact. L'étape de déploiement ne reconstruit jamais : c'est la même image, bit pour bit, qui a été testée et qui part en production.

9. Exploitation : observabilité et procédures

- Accès aux journaux par service et par instance.
- Capacité à répondre en une commande : quels services tournent, sur quelles machines, en combien d'instances, dans quelle version.
- Cinq procédures écrites, en dix lignes maximum chacune : déploiement initial, mise à jour, retour arrière, arrêt et redémarrage du cluster, restauration des données.

Point d'attention. Une bonne plateforme n'est pas celle qui ne tombe jamais, mais celle qui se remet debout vite et proprement. Une procédure qu'un camarade ne peut pas suivre seul n'est pas une procédure.

10. Anticiper les évolutions

Vous occupez un rôle DevSecOps. En cours de projet, une équipe de développement se rendra compte qu'il manque un service. C'est systématique, et cela fait partie du sujet.

Votre infrastructure doit permettre d'ajouter un huitième service sans toucher aux sept autres : conventions de nommage, réseaux et secrets pensés à l'avance, fichiers de déploiement factorisés, point d'entrée qui n'a pas besoin d'être modifié pour router un service de plus.

Vérifié en soutenance. Un service supplémentaire vous sera demandé pendant la démonstration. L'objectif est de l'ajouter et de le déployer en moins de dix minutes. C'est le meilleur test de la qualité de votre

infrastructure, et il ne se prépare pas à l'avance : seule la façon dont vous avez construit votre stack permet d'y arriver.

11. Livrables

Livrable	Description
Construction du cluster	Procédure ou scripts permettant de recréer les trois machines et le cluster depuis zéro.
Déploiement	Fichiers déclaratifs de déploiement, réseaux, secrets et configurations nécessaires.
Documentation	Services, dépendances, flux, hypothèses et limites assumées. Un schéma d'architecture.
Procédures	Les cinq procédures de la section 9.
Automatisation	Définition du flux construction, publication, déploiement, avec une preuve d'exécution.
Registry	Images versionnées effectivement publiées et récupérables par les trois machines.

12. Vérifications : les dix scénarios

Préparez-les tous. La soutenance en joue quatre, tirés au sort, plus l'ajout d'un service.

#	Scénario	Preuve attendue
1	Les trois machines forment un seul cluster	Une liste de nœuds montrant trois machines prêtes et actives, et celle qui dirige.
2	Arrêt puis redémarrage complet du cluster	Tout s'éteint, tout revient : services relancés, données intactes, sans intervention hors documentation.
3	Déploiement depuis zéro	Le déploiement se fait en suivant votre documentation, sans que vous interveniez autrement que par les commandes qu'elle contient.
4	Contrôle de l'exposition	Un seul port répond depuis l'extérieur. La base et le bus ne répondent pas.
5	Placement cohérent	La base est sur la machine prévue, les services sans état sur les autres, et la contrainte est visible dans le fichier.
6	Montée en charge d'un service sans état	Le nombre d'instances augmente et la charge se répartit effectivement.
7	Mise à jour sans interruption perceptible	Des requêtes émises pendant la mise à jour aboutissent toutes.
8	Version défectueuse puis retour arrière	L'échec est constaté, le retour arrière effectué et chronométré.
9	Panne du plan de données	Après arrêt et relance, la donnée est toujours là, ou la restauration est appliquée.
10	Ajout d'un service imprévu	Un huitième service est ajouté et déployé pendant la soutenance.

13. La soutenance

Dix minutes par étudiant, sur votre cluster, en direct.

Temps	Ce qui se passe
2 min	Vous détruisez la stack et vous la redéployez en suivant votre documentation. L'enseignant ne touche pas au clavier.
3 min	Deux scénarios tirés au sort parmi les dix.

Temps	Ce qui se passe
3 min	L'enseignant demande un huitième service, routé et déployé.
2 min	Questions. L'enseignant choisit une ligne de votre fichier et vous demande ce qui casse si elle disparaît. Vous répondez, puis on l'essaie.

Dit une fois, et c'est vrai. Un « je ne sais pas » assumé vaut mieux qu'une explication inventée. L'exercice de prédiction porte sur votre fichier et votre cluster : il n'est pas préparable à l'avance, et c'est volontaire.

14. Barème

Critère	Points	Ce qui est évalué
Cluster et cycle de vie	5	Scénarios 1 et 2 : trois machines reliées, arrêt et redémarrage maîtrisés
Déploiement reproductible	4	Scénario 3, sans assistance
Réseaux, placement et exposition	3	Scénarios 4 et 5
Disponibilité et retour arrière	4	Scénarios 6, 7 et 8
Persistance et restauration	3	Scénario 9, procédure de restauration testée
Livraison automatisée et registry	3	Images versionnées, flux automatisé, identifiants protégés
Documentation et procédures	3	Les cinq procédures, utilisables par un tiers
Évolutivité	3	Scénario 10
Compréhension à l'oral	2	Prédiction juste, justification de vos propres choix

Total sur 30, ramené sur 20. Trois points de bonus possibles : indicateurs de ressources ou d'erreurs exposés, analyse de vulnérabilités bloquante dans le flux de livraison, ou chiffrement du point d'entrée.

Éliminatoire. Un identifiant présent dans l'historique Git. Un secret publié une fois est compromis, même supprimé au commit suivant. Vérifiez avant de rendre.

Annexe A : contrats imposés

Implémentez exactement ces routes. Toute réponse est en JSON. Aucune autre route n'est attendue et aucune ne sera valorisée.

Service	Route	Comportement
tous	GET /health	200 et un corps contenant le nom du service, sa version et le nom d'hôte du conteneur.
comptes	POST /comptes	Insère { pseudo } en base, renvoie 201 et l'identifiant créé.
comptes	GET /comptes/{id}	Renvoie le compte, ou 404.
publications	POST /publications	Vérifie l'auteur auprès de comptes, insère en base, publie un événement sur le bus, renvoie 201.
publications	GET /fil	Renvoie les 20 dernières publications. Le résultat est mis en cache 30 secondes.
worker-medias	aucune route hors /health	Consomme les événements du bus et écrit une trace pour chaque publication reçue.

Format de l'événement publié sur le bus : un objet JSON contenant l'identifiant de la publication, l'identifiant de l'auteur et un horodatage. Rien d'autre.

Le nom d'hôte dans /health n'est pas décoratif. C'est lui qui vous permettra de prouver la répartition de charge au scénario 6. Ne l'omettez pas.

Annexe B : où chaque exigence a été traitée en cours

Aucune exigence de ce sujet ne repose sur une notion qui n'a pas été vue. En cas de blocage, la colonne de droite indique où chercher.

Exigence du sujet	Séance
Images, Dockerfile, cache de construction	TD 1
Réseaux, persistance, sonde de santé, dépendances	TD 2
Créer le cluster, relier les nœuds, étiquettes	TD 3
Placement, réseaux internes, exposition d'un port	TD 3
Secrets et configurations	TD 3
Mise à jour progressive et retour arrière	TD 3 et TD 4
Registry, tags immuables, identifiants de registry	TD 4
Livraison automatisée, déploiement déclenché	TD 4
Journaux, état des services, outil d'administration	TD 4
Sauvegarde et restauration de la base	Procédure à écrire : un export depuis le conteneur, puis un import. Aucune notion nouvelle.

Annexe C : hors périmètre

Ces sujets ne sont pas demandés ils sont en bonus. Ils sont listés pour éviter que vous y consacriez trop de temps.

- Chiffrement du point d'entrée et gestion de certificats : bonus uniquement, jamais exigé.
- Migrations de schéma de base de données : créez les tables au premier démarrage, cela suffit.
- Métriques détaillées, tableaux de bord et mesure de latence : bonus uniquement.
- Authentification réelle, mots de passe utilisateurs, sessions. Un pseudonyme suffit à identifier un compte.
- Stockage objet, quotas et cycle de vie des médias : le worker écrit une trace, cela suffit à démontrer l'asynchrone.
- Haute disponibilité du plan de données. Une seule instance de base, correctement placée et sauvegardée, est ce qui est attendu.
- Interface graphique, tests automatisés de l'application, qualité du code applicatif.